tetFi

Decentralized Lottery Protocol

Trustless · Multi-Mode · On-Chain Settlement

3	0%	968	5%
Lottery Modes	Custody Risk	BOT Chain ID	Platform Fee

Whitepaper · Version 1.0 · 2026
Built on BOT Chain Testnet · rpc.bohr.life · scan.bohr.life

Table of Contents

1. Abstract
2. Introduction & Problem Statement
3. tetFi Protocol Overview
4. Lottery Mechanics
    4.1 Ticket-Based Mode
    4.2 Pool-Based Mode
    4.3 Raffle Mode
5. Smart Contract Architecture
    5.1 Contract State Machine
    5.2 Randomness & Fairness
    5.3 Fee Structure
6. BOT Chain Integration
7. Frontend & User Experience
8. Security Considerations
9. Tokenomics & Fee Distribution
10. Roadmap
11. Risk Factors
12. Conclusion
13. Legal Disclaimer

1. Abstract

tetFi is a fully decentralized lottery protocol deployed on BOT Chain (Chain ID 968). It enables trustless, transparent, and permissionless lotteries with on-chain prize settlement, eliminating the need for any trusted third party. The protocol supports three distinct participation mechanics — Ticket-Based, Pool-Based, and Raffle — each governed by immutable smart contract logic written in Solidity. All funds, draw logic, and winner payouts are executed entirely on-chain. tetFi is designed to be the foundational lottery infrastructure layer for the BOT Chain ecosystem.

■ tetFi smart contracts hold no admin keys over funds. Prize settlement is automatic and irreversible once triggered by the draw function.

3. tetFi Protocol Overview

tetFi is a single-contract, multi-round lottery system. Each round is an isolated financial and logical unit: it has its own mode, price, participant list, jackpot, and winner. Rounds are sequential — a new round cannot begin until the previous one is fully settled.

Protocol Parameters

Network	BOT Chain Testnet — Chain ID 968
RPC Endpoint	https://rpc.bohr.life
Block Explorer	https://scan.bohr.life
Language	Solidity ^0.8.20
Toolchain	Foundry (forge, cast, anvil)
Frontend	Vanilla HTML / CSS / JavaScript + ethers.js v6
Fee	5% of jackpot per round (configurable up to 20% at deploy)
Modes	Ticket-Based, Pool-Based, Raffle
Admin	Contract owner (deployer) manages round lifecycle only

Round Lifecycle

Every lottery round moves through a deterministic four-state machine:

State	Description	Who Triggers
Inactive	No round exists or previous round closed	—
Active	Round open — players can buy tickets / contribute	Owner: startRound()
Drawing	Round closed to new entries, awaiting draw	Owner: closeRound()
Closed	Winner drawn, prize sent, round archived	Owner: drawWinner()

4. Lottery Mechanics

tetFi supports three participation modes. The active mode is set by the owner at round creation and cannot be changed mid-round. Each mode targets a different participation profile and risk preference.

4.1 Ticket-Based Mode

The classic lottery experience. Players purchase one or more numbered tickets at a fixed price per ticket. At draw time, a single ticket index is selected at random and the holding address receives the entire jackpot minus the platform fee.

Entry Function	buyTickets(uint256 count)
Payment	ticketPrice * count (exact, reverts on mismatch)
Odds	Proportional to tickets held vs. total sold
Max Tickets	Optional cap set at round start (0 = unlimited)
Winner Selection	Random index in tickets[] array

4.2 Pool-Based Mode

Participants contribute any amount above a minimum threshold. The jackpot grows with each contribution. At draw time, one contributor address is selected at random — regardless of contribution size. This mode prioritizes access over proportional odds: every wallet has an equal chance of winning.

Entry Function	contribute()
Payment	Any amount >= ticketPrice (used as minimum threshold)
Odds	1-in-N where N = number of unique contributors
Winner Selection	Random index in poolParticipants[] array
Use Case	Community pools, equal-chance jackpots

4.3 Raffle Mode

Raffle mode layers a supply cap on top of the ticket mechanic. The owner sets a maxTickets value; once all tickets are sold the round is guaranteed to produce a winner. This creates a predictable, time-limited sale window and a guaranteed draw without the admin needing to manually close the round.

Entry Function	buyTickets(uint256 count)
Supply Cap	maxTickets — enforced on-chain, reverts when sold out
Odds	Proportional to tickets held vs. maxTickets
Draw Trigger	Manual (owner calls closeRound) or auto at cap
Use Case	Limited-edition drops, community fundraisers

5. Smart Contract Architecture

The tetFi protocol is implemented as a single Solidity contract targeting the EVM-compatible BOT Chain. The contract is intentionally monolithic to minimize attack surface and reduce external call risk.

Core Storage Layout

The following mappings form the protocol's on-chain data model:

rounds	mapping(uint256 => Round) — all historical rounds by ID
tickets	mapping(uint256 => address[]) — ticket holders per round
playerTickets	mapping(uint256 => mapping(address => uint256)) — per-player ticket count
poolContrib	mapping(uint256 => mapping(address => uint256)) — contribution amount per player
poolParticipants	mapping(uint256 => address[]) — unique pool contributors
winHistory	WinRecord[] — append-only ledger of all past winners

5.1 Contract State Machine

The Round struct encodes state as a uint8 enum (Inactive / Active / Drawing / Closed). State transitions are enforced by require() guards and cannot be skipped or reversed. The drawWinner() function additionally checks the settled boolean flag to prevent double-draws in the event of unexpected re-entry.

5.2 Randomness & Fairness

■ The current implementation uses block.prevrandao combined with block.timestamp and participant count as entropy sources. This is sufficient for low-to-medium value rounds but can be biased by validators on high-value rounds.

The winning index is derived as:

6. BOT Chain Integration

tetFi is built natively for BOT Chain, an EVM-compatible Layer 1 designed for African and emerging-market DeFi applications. BOT Chain's low transaction costs make micro-lottery participation economically viable at ticket prices as low as 0.001 BOT.

Chain Name	BOT Chain Testnet
Chain ID	968
Native Currency	BOT (18 decimals)
RPC URL	https://rpc.bohr.life
Block Explorer	https://scan.bohr.life
Website	https://botchain.ai
Consensus	EVM-compatible PoS
Block Time	~2s (estimated)
Finality	Single-slot (optimistic)

Adding BOT Chain to MetaMask

1. Open MetaMask → Settings → Networks → Add Network
2. Network Name: BOT Chain Testnet
3. RPC URL: https://rpc.bohr.life
4. Chain ID: 968
5. Currency Symbol: BOT
6. Block Explorer: https://scan.bohr.life
7. Save and switch to BOT Chain Testnet

✓ The tetFi frontend automatically prompts wallet_addEthereumChain for first-time visitors, so manual network configuration is not required.

7. Frontend & User Experience

The tetFi interface is a single self-contained HTML file with no build step, no framework dependency, and no backend. It can be served from any static host, IPFS, or opened directly from disk.

Interface Sections

Section	Description
Jackpot Hero	Live jackpot display, round state pill, countdown timer, mode and ticket count
Buy Tickets	Ticket counter with live odds bar, cost calculator, one-click purchase
Pool Contribute	Contribution input, minimum display, pool mode auto-detection
Past Winners	Historical draw table with explorer links, mode badges, prize amounts
Admin Panel	Owner-gated: start round, configure mode/price/cap, close, draw, withdraw fees
Footer	BOT Chain logo, links to botchain.ai, block explorer, and RPC endpoint

Wallet Connection

The connect modal supports three wallet types:

Wallet	Method	Notes
MetaMask	window.ethereum injection	Auto-adds BOT Chain if absent
WalletConnect v2	wc: URI + QR code	Requires WC project ID (free at cloud.walletconnect.com)
Coinbase Wallet	window.ethereum injection	Same flow as MetaMask

The frontend also maintains a read-only JsonRpcProvider pointed at rpc.bohr.life so jackpot and round state load immediately on page open without requiring a wallet connection.

8. Security Considerations

tetFi is designed with a minimal attack surface. The following vectors are documented along with their mitigations:

Vector	Risk Level	Mitigation
RNG manipulation	Medium	block.prevrandao; upgrade to VRF in v2
Reentrancy (payout)	Low	CEI pattern; consider OpenZeppelin ReentrancyGuard
Owner key compromise	High	Use multisig; consider Ownable2Step
Front-running draw	Low	commit-reveal or VRF eliminates this in v2
Zero-participant draw	None	require(participants > 0) guard
Double settlement	None	settled boolean flag + state check
Fee overflow	None	Fee capped at 20% in constructor
ETH stuck	None	withdrawFees() always available to owner

■ The tetFi v1 contract has NOT been formally audited. Do not use mainnet or high-value deployments until a professional audit is completed.

9. Tokenomics & Fee Distribution

tetFi does not issue a native token. The protocol operates entirely in BOT, the native currency of BOT Chain. This design choice eliminates token speculation risk and simplifies participation to a single asset.

Fee Flow

Jackpot Source	All ticket / contribution payments sent to contract
Fee Deduction	5% deducted from jackpot at settlement time
Winner Transfer	95% sent directly to winner address in drawWinner()
Fee Accumulation	Fees held in contract.accumulatedFees
Fee Withdrawal	Owner calls withdrawFees(to) to collect
Future Use	Protocol development, audits, ecosystem grants

Example Round (100 BOT jackpot, 5% fee)

Item	Amount (BOT)	Notes
Total jackpot collected	100.0000	Sum of all ticket payments
Platform fee (5%)	5.0000	Retained in contract
Winner prize (95%)	95.0000	Sent in same tx as draw
Gas cost (winner)	~0.001	Estimated on BOT Chain
Net to winner	~94.999	Prize minus gas

10. Roadmap

Q1 2026 — v1.0 Launch (October 2026)

- Deploy Lottery.sol on BOT Chain Testnet
- Release open-source frontend (single HTML file)
- Support Ticket, Pool, and Raffle modes
- WalletConnect v2 + MetaMask + Coinbase Wallet integration
- Block explorer integration (scan.bohr.life)

Q2 2027 — v1.1 Security & UX

- Third-party smart contract audit
- OpenZeppelin ReentrancyGuard integration
- Ownable2Step for safer ownership transfers
- Mobile-optimized UI improvements
- IPFS / Arweave static hosting

Q3 2027 — v2.0 Fair Randomness

- VRF oracle integration (Chainlink or native BOT Chain VRF)
- Commit-reveal scheme as VRF fallback
- Automated round scheduling (time-locked rounds)
- Multi-token support (ERC-20 jackpots)

Q4 2027 — v2.1 Ecosystem

- DAO governance for fee parameters
- Referral module (on-chain affiliate tracking)
- SDK / npm package for third-party integrations
- BOT Chain mainnet deployment
- Multi-chain expansion (Celo, Stellar Soroban)

11. Risk Factors

Smart Contract Risk

Despite careful design, undiscovered bugs may exist in the contract. Users participate at their own risk. A professional audit is planned for v1.1.

Randomness Risk

v1 uses block-level entropy which can theoretically be biased by validators in high-value scenarios. VRF integration is planned for v2.

Regulatory Risk

Lottery regulations vary by jurisdiction. Users are responsible for compliance with local laws. tetFi operates as neutral infrastructure only.

Liquidity Risk

Jackpot funds are locked in the contract until a round is settled. In the event of an unresolvable contract bug, funds could be inaccessible.

Network Risk

BOT Chain Testnet may experience outages, reorganizations, or parameter changes that affect contract behavior. Mainnet deployment carries its own risks.

Owner Key Risk

The contract owner has the ability to start, close, and draw rounds. Compromise of the owner key could allow manipulation. A multisig is strongly recommended.

12. Conclusion

tetFi demonstrates that a fully functional, trustless lottery protocol can be built and deployed on BOT Chain with a compact Solidity contract and a zero-dependency frontend. By eliminating custodial risk, automating prize settlement, and publishing all source code openly, tetFi establishes a foundation for transparent on-chain gaming infrastructure in the BOT Chain ecosystem.

The three lottery modes — Ticket, Pool, and Raffle — give operators the flexibility to design experiences ranging from classic numbered-ticket draws to community pool events with equal-odds participation. The protocol's open architecture means any developer can fork, extend, or integrate tetFi into their own application.

With planned upgrades to verifiable randomness, DAO governance, and multi-chain expansion, tetFi is positioned to grow alongside the BOT Chain ecosystem and serve as the canonical decentralized lottery protocol for the network.

Built on BOT Chain · rpc.bohr.life · scan.bohr.life · botchain.ai

13. Legal Disclaimer

This whitepaper is for informational purposes only and does not constitute financial, legal, or investment advice. Participation in the tetFi lottery protocol involves financial risk, including the possible loss of all contributed funds. Smart contracts may contain bugs or vulnerabilities not identified at time of publication. The tetFi protocol has not been formally audited as of the date of this document.

Users are solely responsible for ensuring that their participation complies with all applicable laws and regulations in their jurisdiction. Lottery, gaming, and gambling activities are regulated or prohibited in certain jurisdictions. It is the user's responsibility to determine the legality of participation before interacting with the tetFi protocol.

The protocol developers and contributors make no warranties, express or implied, regarding the accuracy, reliability, or completeness of information contained in this whitepaper. All statements regarding future development, roadmap items, and planned features are forward-looking and subject to change without notice.

tetFi is experimental software deployed on a testnet. Use at your own risk.